

REACT · VITE · DO ZERO AO AVANÇADO

React na Prática: Ferramenta de Loterias

Aprenda React construindo, do zero, um app offline de loterias: gerar jogos, fechamentos matemáticos e conferir resultados

Componentes, estado, persistência offline, PWA, gráficos e um módulo ilustrado de algoritmos — combinações e o fechamento de loteria, passo a passo.

Curso completo · React 19 · Vite · PWA · Edição 2026

Sumário

Prefácio — React do zero, construindo uma ferramenta de verdade

Módulo 1 — A Fundação

Capítulo 1 — SPA, PWA e o modelo do React

Capítulo 2 — Ambiente e a estrutura do projeto

Capítulo 3 — JSX e estado: a grade de dezenas ganha vida

Módulo 2 — Arquitetura e estilo

Capítulo 4 — Componentes reutilizáveis e CSS Modules

Capítulo 5 — Temas com Context e variáveis CSS

Módulo 3 — As telas de loteria

Capítulo 6 — Navegação com React Router e SEO

Capítulo 7 — Montando e salvando jogos

Capítulo 8 — Listando e removendo jogos

Módulo 4 — Dados, persistência e offline

Capítulo 9 — Persistência local com localStorage

Capítulo 10 — Buscando resultados e conferindo jogos

Capítulo 11 — PWA: instalável e offline

Módulo 5 — Os algoritmos da loteria

Capítulo 12 — Combinações: a base da matemática lotérica

Capítulo 13 — Fechamento: jogar poucos com garantia

Capítulo 14 — Estatísticas e gráficos

Módulo 6 — Conteúdo e publicação

Capítulo 15 — Um blog dentro do app

Capítulo 16 — O espaço de anúncios

Capítulo 17 — Build, deploy e o próximo passo

Prefácio — React do zero, construindo uma ferramenta de verdade

Aprende-se a programar construindo. Neste livro, você vai construir do zero uma aplicação real e sofisticada — uma **ferramenta de loterias** — e, no caminho, aprender React dos fundamentos aos temas avançados. Ao fim, você terá um app que funciona **offline**, gera jogos, calcula **fechamentos matemáticos**, confere resultados e mostra estatísticas — e, mais importante, o entendimento para construir qualquer aplicação web moderna.

Por que uma ferramenta de loterias

Porque ela é rica o suficiente para ensinar React inteiro. Um app de loterias precisa de: interface interativa (a grade de dezenas), estado complexo (jogos selecionados), persistência que funciona sem internet (offline-first), consumo de API (resultados oficiais), gráficos (estatísticas), navegação com URLs amigáveis, temas, e — o mais interessante — **algoritmos de verdade** (combinações e fechamento). É um curso de React completo disfarçado de app útil.

O que vamos construir

Um app chamado **LMania**, com quatro loterias (Lotofácil, Lotomania, Mega-Sena, Quina). As funcionalidades, todas passo a passo:

- **Gerar e salvar jogos**, escolhendo dezenas numa grade interativa.
- **Fechamentos matemáticos** (desdobramentos que reduzem o custo com garantia de prêmio).
- **Conferir** jogos contra o resultado oficial da Caixa.
- **Estatísticas** com gráficos (pares/ímpares, frequência de dezenas).
- **Funcionar offline** e ser instalável como app (PWA).
- **Blog** de conteúdo, temas claro/escuro, e um espaço para anúncios.

Como o livro é organizado

O livro sobe uma escada, do básico ao avançado:

1. **Fundação** — SPA, Vite, e os pilares do React (componentes e estado).
2. **Arquitetura e estilo** — componentes reutilizáveis, CSS isolado, temas.
3. **As telas de loteria** — navegação, a grade de dezenas, salvar e listar jogos.
4. **Dados e persistência** — offline-first, buscar resultados, conferir, PWA.
5. **Os algoritmos** — combinações e o fechamento de loteria, explicados com diagramas.
6. **Conteúdo e publicação** — blog, anúncios, SEO e deploy.

COMO LER OS BOXES

Dica (um atalho), **Cuidado** (uma armadilha comum), **Importante** (um princípio para decorar), **Nota** (aprofundamento) e **Na prática** (como aplicamos ao LMania).

Pré-requisitos

Você precisa saber o básico de JavaScript (variáveis, funções, arrays, loops). Não precisa saber React — ensinamos do zero. Não precisa saber teoria de algoritmos — no Módulo 5, desenhamos o fechamento no papel antes de codificar. Quem começa do início e lê até o fim, reproduz tudo.

Vamos construir — e entender cada peça.

Módulo 1 — A Fundação

Todo prédio precisa de alicerce. Aqui você entende o que é uma aplicação moderna (uma SPA, e um PWA), monta o ambiente com o Vite, e aprende os dois pilares do React: **componentes** (as peças da tela) e **estado** (a memória que a faz viver). Ao fim, você terá o esqueleto do LMania rodando no navegador.

Capítulo 1 — SPA, PWA e o modelo do React

1.1 De páginas a aplicativos

No passado, um site era um monte de páginas HTML: clicou num link, a tela ficou branca, o navegador baixou outra página. Para uma ferramenta interativa — onde você seleciona dezenas, gera jogos, confere resultados sem esperar a tela recarregar — isso não serve.

Hoje construímos **SPAs** (aplicações de página única): o navegador carrega o app uma vez e, a partir daí, a tela muda sozinha em resposta às ações, sem recarregar. E vamos além: o LMania será um **PWA** (*Progressive Web App*) — instalável na tela inicial do celular, com ícone próprio, e funcionando **offline**. O usuário nem vai perceber que é um site.

1.2 O modelo mental do React

O **React** constrói interfaces com uma ideia central que guia o livro inteiro:

> **Você não manda a tela mudar. Você muda os dados, e a tela se atualiza sozinha.**

É a programação **declarativa**. No JavaScript tradicional, você daria ordens passo a passo: "pegue esta bolinha, pinte de laranja, adicione à grade". No React, você **descreve** como a tela deve ser para um estado (ex.: "as dezenas selecionadas ficam laranja") e o React aplica as diferenças quando o estado muda. Numa grade de 60 dezenas que o usuário clica sem parar, essa inversão é o que evita o caos.



Figura 1.1 — No React, a grade de dezenas é um reflexo do estado "dezenas selecionadas". Você muda o estado; a grade reage.

1.3 Componentes: a tela em peças

O React organiza a interface em **componentes** reutilizáveis. No LMania teremos: uma `Bola` (a bolinha de uma dezena), uma `GradeDezenas`, um `CardLoteria`, uma barra de navegação, um painel de resultados. Em vez de uma página gigante de HTML, montamos a tela encaixando componentes — como peças de Lego. Precisou da mesma bolinha em três telas? É o mesmo componente. Mudou o visual dela? Muda num lugar só.

1.4 O que você vai precisar

- **Node.js** (versão LTS) — roda as ferramentas.
- **VS Code** — o editor (gratuito).
- **Um navegador** — para ver e depurar.
- **Vontade de digitar o código** — digitar fixa o aprendizado; não copie tudo.

> **IMPORTANTE — GRAVE O MODELO MENTAL ANTES DA SINTAXE**
>
> Antes de qualquer comando, grave: no React, a **fonte da verdade** são os dados (o estado), não a tela. A tela reflete os dados. Se algo precisa mudar na tela, você muda o dado — nunca "manda a tela mudar" como no JavaScript antigo. Quase todo erro de iniciante vem de esquecer isso. No LMania, "quais dezenas estão selecionadas" será um estado, e a grade colorida será apenas o reflexo dele.

No próximo capítulo, montamos o ambiente com o Vite e vemos o esqueleto do LMania nascer.

Capítulo 2 — Ambiente e a estrutura do projeto

2.1 Criando o projeto com Vite

Usamos o **Vite**, a ferramenta moderna para React — sobe em milissegundos e atualiza a tela quase instantaneamente. Confira o Node primeiro (`node --version` , v20+), e crie o projeto:

```
npm create vite@latest lmania-react -- --template react
cd lmania-react
npm install
npm run dev
```

Abra `http://localhost:5173` . React rodando em segundos.

2.2 Limpando e preparando o terreno

Limpe o `src/App.jsx` (deixe um `<h1>LMania</h1>`) e configure o `src/index.css` com um reset e os **tokens** — as variáveis de design que darão identidade e tema ao app:

```
/* src/index.css */
* { margin: 0; padding: 0; box-sizing: border-box; }

:root {
  --bg: #080c14;          --surface: #0e1420;   --card: #131b2e;
  --border: rgba(255,255,255,.07);  --text: #e8edf5;  --text-muted: #94a3b8;
  --accent: #f97316;      --primary: #3b82f6;   --success: #22c55e;  --danger: #ef4444;
  --sp-2: 8px;  --sp-3: 12px;  --sp-4: 16px;  --sp-6: 24px;
  --radius: 16px;
}

body {
  font-family: -apple-system, "Segoe UI", Roboto, sans-serif;
  background: var(--bg); color: var(--text); min-height: 100dvh;
}
```

DICA — TOKENS DESDE O COMEÇO

Definir as cores e espaçamentos como variáveis (`--accent` , `--sp-4`) já no início parece exagero, mas é o que vai permitir, no Módulo 2, ter **temas** (claro/escuro) e um visual consistente sem repetir cores. Os componentes vão usar `var(--accent)` , nunca um `#f97316` solto. Mude a variável, muda o app inteiro.

2.3 A planta do app

Um app do porte do LMania precisa de organização. Dentro de `src/`, esta é a estrutura que vamos preencher:

```
src/
├── components/   # atoms (Bola, Botao), molecules (CardLoteria), organisms (Navbar, Grade)
├── pages/        # as telas (Home, Loteria, Blog, Configurações)
├── context/      # estado compartilhado (tema)
├── hooks/        # lógica reutilizável (useLocalStorage...)
├── services/     # lógica pura: loterias (config, combinações, fechamento), storage, resultados
└── assets/       # ícones, dados (tabelas de cobertura)
```

> **IMPORTANTE — A CAMADA `SERVICES/` É O CÉREBRO**
>
> Repare na pasta `services/`. Nela vai morar toda a **lógica pura** do LMania — os algoritmos de loteria, o acesso ao storage, a busca de resultados — **sem uma linha de React**. Isso é intencional: lógica pura é testável, reutilizável e não quebra quando a interface muda. É a decisão de arquitetura mais importante do livro, e vamos voltar a ela sempre. A regra: se calcula ou busca dados, vai para `services/`; se desenha a tela, vai para `components/`.

2.4 A configuração das loterias

O LMania tem quatro loterias, cada uma com sua matemática. Centralizamos isso num único arquivo de dados — a fonte da verdade que todas as telas vão consultar:

```
// src/services/loterias/config.js
export const LOTERIAS = [
  { nome: 'Lotofácil', url: 'lotofacil', max: 25,  escolhe: 15, sorteia: 15, accent: '#a855f7' },
  { nome: 'Lotomania',url: 'lotomania', max: 100, escolhe: 50, sorteia: 20, accent: '#ef4444' },
  { nome: 'Mega-Sena',url: 'mega-sena', max: 60,  escolhe: 6,  sorteia: 6,  accent: '#22c55e' },
  { nome: 'Quina',    url: 'quina',     max: 80,  escolhe: 5,  sorteia: 5,  accent: '#3b82f6' },
];
```

- `max` = quantas dezenas existem no volante; `escolhe` = mínimo de um jogo; `sorteia` = quantas saem; `accent` = a cor da loteria. Uma tela parametrizada por essa config serve às quatro — em vez de quatro telas quase iguais.

O ambiente e a planta estão prontos. No próximo capítulo, entendemos **como o React desenha** (o JSX), antes de construir a primeira peça de verdade: a grade de dezenas.

Capítulo 3 — JSX e estado: a grade de dezenas ganha vida

3.1 JSX: descrevendo a interface

O React usa **JSX** — uma sintaxe parecida com HTML, dentro do JavaScript, que descreve a tela. Três regras evitam os erros mais comuns: um componente retorna **uma** raiz (use `<>...</>` para agrupar); usa-se `className` em vez de `class`; e eventos em camelCase (`onClick`). O poder está em misturar JavaScript com **chaves** `{}` — inclusive gerar listas com `.map()`:

```
// gerar as bolinhas de 1 até max
{Array.from({ length: 60 }, (_, i) => i + 1).map((numero) => (
  <button key={numero} className="bola">{numero}</button>
))}
```

Esse trecho já é o esqueleto da grade da Mega-Sena: 60 botões numerados, gerados por um `.map()`. Repare na prop `key={numero}` — obrigatória em listas, ela dá a cada item um identificador estável para o React.

3.2 `useState`: a memória da seleção

A grade precisa lembrar **quais dezenas o usuário selecionou**. Essa memória é o **estado**, criado com `useState`. Para um conjunto de dezenas, um `Set` é perfeito (não repete, e testa "contém?" rápido):

```jsx
import { useState } from 'react';

function GradeDezenas({ max }) {
  const [selecionadas, setSelecionadas] = useState(new Set());

  function alternar(numero) {
    setSelecionadas((atual) => {
      const nova = new Set(atual);          // cria um Set NOVO (imutabilidade!)
      nova.has(numero) ? nova.delete(numero) : nova.add(numero);
      return nova;
    });
  }

  return (
    <div className="grade">
      {Array.from({ length: max }, (_, i) => i + 1).map((n) => (
        <button
          key={n}
          className={selecionadas.has(n) ? 'bola selecionada' : 'bola'}
          onClick={() => alternar(n)}
        >
          {n}
        </button>
      ))}
    </div>
  );
}
```

Clique numa bolinha: `alternar` cria um `Set` novo, adiciona ou remove o número, e chama `setSelecionadas`. O React re-renderiza, e a bolinha ganha (ou perde) a classe `selecionada`. Você não pintou nada na mão — descreveu "selecionada fica laranja" no CSS, e mudou o dado.

> **IMPORTANTE — POR QUE CRIAR UM `SET` NOVO**
>
> Repare que criamos `new Set(atual)` em vez de mexer no `Set` existente. O React decide se atualiza a tela comparando **referências**: se você mudasse o mesmo `Set`, a referência seria a mesma, e o React pensaria "nada mudou". Criar uma cópia nova é o que faz a tela reagir. Vale para `Set`, arrays (`[...arr]`) e objetos (`{...obj}`): sempre uma caixa nova. Esse é o erro nº 1 de quem começa — grave-o.

3.3 O estado deriva a interface inteira

Com o estado `selecionadas`, tudo na tela deriva dele — sem código extra:

```
const quantas = selecionadas.size;                    // "6 de 6 escolhidas"
const completo = quantas === sorteia;                 // habilita o botão salvar
const podeSalvar = quantas >= escolhe;

<p>{quantas} de {sorteia} dezenas</p>
<Botao texto="Salvar jogo" desabilitado={!podeSalvar} />
```

O contador, o aviso, o botão habilitado — tudo é calculado a partir do estado, a cada render. Você nunca sincroniza "na mão" o contador com a grade: os dois vêm da mesma fonte, então nunca divergem. É a beleza do modelo declarativo.

3.4 `useEffect`: o mundo de fora

O `useState` cuida do que está dentro do componente. Para o que está fora — buscar dados, ler o storage, ligar um timer — usamos o `useEffect`, com um array de dependências que controla quando ele roda (`[]` = uma vez, na montagem). Vamos usá-lo muito nos módulos de dados. Por ora, guarde a forma:

```
useEffect(() => {
  // roda depois da tela aparecer
  return () => { /* limpeza, quando a tela sai */ };
}, [dependencias]);
```

> **DICA — O `SET` PARA SELEÇÃO É UMA BOA ESCOLHA**
>
> Usar um `Set` (em vez de um array) para as dezenas selecionadas tem duas vantagens: testar "está selecionada?" é instantâneo (`selecionadas.has(n)`), e não há risco de duplicar uma dezena. Ao salvar o jogo, converta para array ordenado: `[...selecionadas].sort((a,b) => a-b)`. Escolher a estrutura de dados certa deixa o código mais limpo — um hábito que vamos cultivar no livro inteiro, até os algoritmos do Módulo 5.

Você já tem a grade interativa — o coração visual do LMania. No próximo módulo, organizamos isso em componentes reutilizáveis e damos ao app estilo e temas.

Módulo 2 — Arquitetura e estilo

Um app cresce sem virar caos quando é bem organizado. Aqui você aprende a construir a interface com peças reutilizáveis (Atomic Design), a estilizá-las sem conflitos (CSS Modules), e a dar ao LMania um sistema de **temas** claro/escuro — tudo apoiado nos design tokens que definimos no Módulo 1.

Capítulo 4 — Componentes reutilizáveis e CSS Modules

4.1 Props e Atomic Design

Um componente vira reutilizável quando recebe dados por **props**. E organizamos os componentes em três níveis (**Atomic Design**): **átomos** (peças mínimas: `Bola`, `Botao`), **moléculas** (átomos combinados: `CardLoteria`), **organismos** (blocos com estado e lógica: `GradeDezenas`, `Navbar`, `PainelResultados`).

Vamos extrair a `Bola` do capítulo anterior num átomo próprio:

```jsx
// src/components/atoms/Bola/Bola.jsx
import estilos from './Bola.module.css';

export default function Bola({ numero, selecionada = false, aoClicar }) {
  return (
    <button
      className={`${estilos.bola} ${selecionada ? estilos.selecionada : ''}`}
      onClick={() => aoClicar?.(numero)}
    >
      {numero}
    </button>
  );
}
```

A `Bola` é "burra": não sabe o que é uma loteria, só desenha um número e avisa quando é clicada (via `aoClicar`). Esse padrão — **dados descem por props, eventos sobem por callbacks** — é a espinha dorsal do React, e o que torna a `Bola` reutilizável em qualquer grade.

4.2 CSS Modules: estilo sem conflito

CSS é global — uma regra `.bola` afetaria toda `.bola` da página. O **Vite** resolve isso com os **CSS Modules**: nomeie o arquivo `Componente.module.css` e importe como objeto. O Vite gera nomes únicos por baixo, isolando cada componente.

```css
/* src/components/atoms/Bola/Bola.module.css */
.bola {
  width: 40px; height: 40px; border-radius: 50%;
  background: var(--card); color: var(--text);
  border: 1px solid var(--border); font-weight: 600; cursor: pointer;
}
.selecionada {
  background: var(--accent); color: #fff; border-color: var(--accent);
}
```

Note que usamos os **tokens** (`var(--card)` , `var(--accent)`) do Módulo 1 — não cores fixas. A bolinha selecionada fica na cor de destaque, seja ela qual for no tema atual.

4.3 Uma molécula: o card de loteria

A Home mostra as quatro loterias numa grade de cards. Cada card é uma molécula que recebe a config e avisa quando é escolhido:

```jsx
// src/components/molecules/CardLoteria/CardLoteria.jsx
import estilos from './CardLoteria.module.css';

export default function CardLoteria({ loteria, aoEscolher }) {
  return (
    <button
      className={estilos.card}
      style={{ '--cor': loteria.accent }}    // passa a cor da loteria via CSS variable
      onClick={() => aoEscolher(loteria)}
    >
      <span className={estilos.nome}>{loteria.nome}</span>
      <span className={estilos.info}>{loteria.escolhe} a {loteria.max} dezenas</span>
    </button>
  );
}
```

> **DICA — PASSANDO VALORES DINÂMICOS AO CSS**
>
> Repare no `style={{ '--cor': loteria.accent }}` : passamos a cor específica da loteria como uma variável CSS inline. No `.module.css` , o card usa `var(--cor)` para a borda ou o brilho. Assim, cada card tem a cor da sua loteria (roxo para Lotofácil, verde para Mega...) com **um** componente e **um** CSS — sem quatro variações. É a mistura elegante de props (React) com variáveis (CSS).

4.4 A grade da Home

A Home monta os cards com um `.map()` sobre a config — a mesma inversão de sempre: dados descrevem, componente desenha:

```jsx
// src/pages/Home/Home.jsx
import { LOTERIAS } from '../../services/loterias/config';
import CardLoteria from '../../components/molecules/CardLoteria/CardLoteria';

export default function Home() {
  const navegar = useNavigate();
  return (
    <div className="grade-loterias">
      {LOTERIAS.map((l) => (
        <CardLoteria key={l.url} loteria={l} aoEscolher={(lot) => navegar(`/${lot.url}`)} />
      ))}
    </div>
  );
}
```

IMPORTANTE — O GANHO CONCRETO DA REUTILIZAÇÃO

Adicionar uma quinta loteria ao LMania? Basta uma linha no arquivo de config — a Home ganha o card automaticamente, a tela de loteria funciona parametrizada, tudo sem escrever componentes novos. É a recompensa de construir com peças (Atomic Design) e centralizar dados (a config). O app cresce por configuração, não por cópia.

Com componentes e estilo prontos, o próximo passo é o sistema de temas — que aproveita os tokens para claro/escuro instantâneo.

Capítulo 5 — Temas com Context e variáveis CSS

5.1 Estado que várias telas compartilham

O tema (as cores do app) é conhecido por **todas** as telas ao mesmo tempo. Passar isso de componente em componente seria um suplício. A solução do React é o **Context** — um jeito de disponibilizar um valor para toda a árvore de componentes, sem passá-lo manualmente.

5.2 Temas como dados

Primeiro, definimos os temas como objetos de dados (em `services/`), cada um com seu conjunto de cores:

```
// src/services/temas.js
export const TEMAS = {
  escuro: { bg: '#080c14', card: '#131b2e', text: '#e8edf5', accent: '#f97316' },
  claro:  { bg: '#f8fafc', card: '#ffffff', text: '#0f172a', accent: '#ea580c' },
};
export const TEMA_PADRAO = 'escuro';
```

5.3 O TemaContext

O Context guarda o tema atual, aplica as cores como **variáveis CSS** no elemento raiz, e persiste a escolha:

```jsx
// src/context/TemaContext.jsx
import { createContext, useContext, useState, useEffect } from 'react';
import { TEMAS, TEMA_PADRAO } from '../services/temas';

const TemaContext = createContext(null);

export function TemaProvider({ children }) {
  const [nome, setNome] = useState(() => localStorage.getItem('lmania_tema') ?? TEMA_PADRAO);

  useEffect(() => {
    const tema = TEMAS[nome] ?? TEMAS[TEMA_PADRAO];
    const root = document.documentElement;
    root.style.setProperty('--bg', tema.bg);       // aplica cada cor como variável
    root.style.setProperty('--card', tema.card);
    root.style.setProperty('--text', tema.text);
    root.style.setProperty('--accent', tema.accent);
    localStorage.setItem('lmania_tema', nome);      // persiste
  }, [nome]);

  return (
    <TemaContext.Provider value={{ nome, trocar: setNome, temas: Object.keys(TEMAS) }}>
      {children}
    </TemaContext.Provider>
  );
}

export const useTema = () => useContext(TemaContext);
```

5.4 Por que isso é tão eficiente

A mágica: como **todos** os componentes usam `var(--bg)`, `var(--text)`, `var(--accent)` no CSS (graças aos tokens do Módulo 1), trocar o tema é só mudar o valor dessas variáveis no `:root`. O app inteiro muda de cor **instantaneamente, sem re-renderizar o React** — o CSS faz o trabalho. Você não injeta `style` em cada componente; muda quatro variáveis, e milhares de elementos reagem.

```jsx
// a tela de configurações troca o tema com uma linha:
const { nome, trocar, temas } = useTema();
{temas.map((t) => (
  <button key={t} onClick={() => trocar(t)}>{t}</button>
))}
```

IMPORTANTE — VARIÁVEIS SEMÂNTICAS, NÃO PIGMENTOS

Nomeamos as variáveis pela **função** (`--bg`, `--text`, `--accent`), não pelo pigmento (`--preto`, `--laranja`). Isso é vital: no tema claro, `--text` vira escuro; no escuro, vira claro. Se a variável se chamasse `--branco`, você teria o absurdo de `color: var(--branco)` pintando de preto. A variável descreve o papel da cor, e é o papel que muda entre temas. Esse cuidado é o que torna o tema claro/escuro elegante em vez de remendado.

5.5 Ligando o Provider

O `TemaProvider` envolve o app no `main.jsx`, para que qualquer tela alcance o tema:

```
// src/main.jsx
import { TemaProvider } from './context/TemaContext';

createRoot(document.getElementById('root')).render(
  <TemaProvider>
    <App />
  </TemaProvider>
);
```

> **DICA — O MESMO PADRÃO SERVE PARA O TAMANHO DE FONTE**
>
> O LMania original também tem controle de tamanho de fonte (acessibilidade). A técnica é idêntica à do tema: mudar uma propriedade no `:root` (`document.documentElement.style.fontSize = '110%'`) e persistir. Como todo o CSS usa unidades relativas (`rem`), a fonte escala proporcionalmente. Você pode envolver isso no mesmo Context de tema, ou num hook `useTamanhoFonte`. Acessibilidade não é opcional — inclua desde já.

5.6 Fecho do módulo

Você tem a arquitetura de interface: componentes reutilizáveis (Atomic Design), estilo isolado (CSS Modules), tokens centralizados e temas via Context. O LMania já tem cara, cores e a grade interativa. No **Módulo 3**, damos a ele telas de verdade — navegação, a tela de cada loteria, e o salvar/listar jogos.

Módulo 3 — As telas de loteria

O app ganha telas de verdade. Neste módulo montamos a navegação (com URLs amigáveis e SEO), a tela de cada loteria com a grade de dezenas, e o fluxo de salvar e listar jogos — a funcionalidade central do LMania, apoiada na persistência local que aprofundamos no módulo seguinte.

Capítulo 6 — Navegação com React Router e SEO

6.1 URLs reais numa SPA

O LMania tem telas — Home, cada loteria, blog, configurações — e queremos URLs de verdade (`/mega-sena`, não `#mega`), para que o usuário possa compartilhar links, o botão voltar funcione, e os buscadores indexem. Isso é o **React Router**:

```
npm install react-router-dom
```

6.2 Declarando as rotas

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';

export default function App() {
  return (
    <BrowserRouter>
      <Layout>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/:loteria" element={<Loteria />} />   {/* mega-sena, quina... */}
          <Route path="/blog" element={<Blog />} />
          <Route path="/blog/:slug" element={<BlogPost />} />
          <Route path="/configuracoes" element={<Configuracoes />} />
        </Routes>
      </Layout>
    </BrowserRouter>
  );
}
```

6.3 Uma rota dinâmica para as quatro loterias

Repare na rota `/:loteria` — os dois-pontos marcam um trecho **variável**. Em vez de quatro telas quase iguais (uma por loteria), temos **uma** que lê o parâmetro e busca a config:

```
// src/pages/Loteria/Loteria.jsx
import { useParams } from 'react-router-dom';
import { LOTERIAS } from '../../services/loterias/config';

export default function Loteria() {
  const { loteria } = useParams();                        // 'mega-sena'
  const config = LOTERIAS.find((l) => l.url === loteria);
  if (!config) return <p>Loteria não encontrada.</p>;
  return <TelaLoteria config={config} />;                  // uma tela, parametrizada
}
```

Uma tela parametrizada substitui quatro. O `useParams()` lê o trecho variável da URL — e a config (Módulo 2) faz o resto.

6.4 Navegando: `Link` e `useNavigate`

Para links, o `<Link>` (muda a URL sem recarregar); após uma ação, o `useNavigate`:

```
import { Link, useNavigate } from 'react-router-dom';
<Link to="/mega-sena">Mega-Sena</Link>

const navegar = useNavigate();
navegar('/quina');    // navega por código
navegar(-1);          // volta uma tela
```

CUIDADO — NUNCA `LOCATION.RELOAD()`

No JavaScript antigo, recarregar a página era comum para "atualizar" a tela. Em React é um erro: destrói o estado e a fluidez. Para mudar de tela, `navegar(...)`; para atualizar dados, mude o estado. Se sentir vontade de recarregar, é sinal de que algo deveria ser estado ou Context.

6.5 SEO por rota

Diferente do Provas (um app interno), o LMania é público — então o SEO importa. Cada tela deve ter um título e uma descrição próprios, para os buscadores. Um hook simples resolve:

```
// src/hooks/useSeo.js
import { useEffect } from 'react';
export function useSeo(titulo, descricao) {
  useEffect(() => {
    document.title = titulo;
    document.querySelector('meta[name="description"]')?.setAttribute('content', descricao);
  }, [titulo, descricao]);
}
// na tela: useSeo('Mega-Sena | LMania', 'Cadastre e confira seus jogos da Mega-Sena.');
```

DICA — SEO EM SPA TEM LIMITES (E SOLUÇÕES)

Como uma SPA renderiza no navegador, alguns buscadores antigos podem não "ver" o conteúdo gerado por JavaScript. Para blogs e páginas públicas, a solução avançada é **SSR** (renderização no servidor, com frameworks como Next.js) ou pré-renderização. Para o LMania, o `useSeo` (título + descrição por rota) já cobre o essencial, e os buscadores modernos executam JavaScript. Comece simples; migre para SSR só se o SEO for crítico e você medir necessidade.

Com a navegação pronta, vamos à tela mais importante: a de uma loteria, onde o usuário monta e salva um jogo.

Capítulo 7 — Montando e salvando jogos

7.1 A tela de loteria

Juntamos as peças: a grade de dezenas (Módulo 1), a config (Módulo 2) e o estado da seleção. A `TelaLoteria` é parametrizada pela config, então serve às quatro loterias:

```jsx
// src/pages/Loteria/TelaLoteria.jsx
import { useState } from 'react';
import Bola from '../../components/atoms/Bola/Bola';
import Botao from '../../components/atoms/Botao/Botao';
import { salvarJogo } from '../../services/storage';
import { useSeo } from '../../hooks/useSeo';
import toast from 'react-hot-toast';

export default function TelaLoteria({ config }) {
  useSeo(`${config.nome} | LMania`, `Monte e salve seus jogos da ${config.nome}.`);
  const [selecionadas, setSelecionadas] = useState(new Set());

  function alternar(n) {
    setSelecionadas((atual) => {
      const nova = new Set(atual);
      if (nova.has(n)) nova.delete(n);
      else if (nova.size < config.sorteia) nova.add(n);   // trava no máximo
      else toast('Você já escolheu o máximo de dezenas.');
      return nova;
    });
  }

  function salvar() {
    const dezenas = [...selecionadas].sort((a, b) => a - b);
    salvarJogo(config.url, dezenas);       // persiste (Módulo 4)
    toast.success('Jogo salvo!');
    setSelecionadas(new Set());            // limpa para o próximo
  }

  const podeSalvar = selecionadas.size >= config.escolhe;

  return (
    <div>
      <h2 style={{ color: config.accent }}>{config.nome}</h2>
      <p>{selecionadas.size} de {config.sorteia} dezenas</p>
      <div className="grade">
        {Array.from({ length: config.max }, (_, i) => i + 1).map((n) => (
          <Bola key={n} numero={n} selecionada={selecionadas.has(n)} aoClicar={alternar} />
        ))}
      </div>
      <Botao texto="Salvar jogo" aoClicar={salvar} desabilitado={!podeSalvar} />
    </div>
  );
}
```

7.2 As regras de negócio no lugar certo

Repare em duas regras que aparecem naturalmente:

- **O máximo de dezenas** (`nova.size < config.sorteia`): não deixa escolher mais que o permitido, com um aviso amigável.
- **O mínimo para salvar** (`podeSalvar = size >= config.escolhe`): o botão só habilita quando há dezenas suficientes.

Ambas derivam do **estado** e da **config** — nada é sincronizado à mão. O contador, o botão, a trava: todos calculados a cada render, a partir da mesma fonte. É o modelo declarativo eliminando bugs de sincronia.

> **IMPORTANTE — A TELA ORQUESTRA, O SERVICE PERSISTE**
>
> A tela chama `salvarJogo(config.url, dezenas)` — uma função do service — e não sabe **como** o jogo é salvo (localStorage? banco? nuvem?). Essa separação é deliberada: a interface cuida da experiência, o service cuida da persistência. Se um dia você trocar o localStorage por sincronização na nuvem, muda só o service; a tela nem percebe. É a arquitetura em camadas pagando dividendos.

7.3 Gerar um jogo aleatório

Um recurso útil: um botão "surpresinha" que preenche dezenas aleatórias. Lógica pura, num service:

```
// src/services/loterias/gerador.js
export function jogoAleatorio(config) {
  const dezenas = new Set();
  while (dezenas.size < config.sorteia) {
    dezenas.add(1 + Math.floor(Math.random() * config.max));
  }
  return new Set(dezenas);
}
// na tela: <Botao texto="Surpresinha" aoClicar={() => setSelecionadas(jogoAleatorio(config))} />
```

> **DICA — LÓGICA PURA FORA DO COMPONENTE**
>
> Repare que `jogoAleatorio` está num service, não dentro do componente. Poderia estar inline, mas separá-lo tem valor: fica testável (você verifica que ele sempre gera a quantidade certa de dezenas únicas), reutilizável, e mantém a tela focada em orquestrar. É a mesma disciplina que, no Módulo 5, vai deixar os algoritmos de fechamento — muito mais complexos — organizados e testáveis.

O usuário monta e salva jogos. Mas para onde eles vão, e como listá-los? É a persistência — mas antes, o capítulo que fecha o fluxo básico: listar e remover os jogos salvos.

Capítulo 8 — Listando e removendo jogos

8.1 A tela de jogos salvos

Salvar sem conseguir ver não serve. A tela de lista mostra os jogos guardados de uma loteria, com a opção de remover. Ela lê os jogos no estado (via o service de storage, que aprofundamos no próximo módulo) e os desenha:

```jsx
// src/pages/Loteria/ListaJogos.jsx
import { useState } from 'react';
import { listarJogos, removerJogo } from '../../services/storage';
import Bola from '../../components/atoms/Bola/Bola';

export default function ListaJogos({ config }) {
  const [jogos, setJogos] = useState(() => listarJogos(config.url)); // lê 1x na inicialização

  function remover(indice) {
    removerJogo(config.url, indice);          // persiste a remoção
    setJogos(listarJogos(config.url));        // atualiza o estado → a tela reage
  }

  if (jogos.length === 0) return <p>Nenhum jogo salvo ainda.</p>;

  return (
    <ul className="lista-jogos">
      {jogos.map((jogo, i) => (
        <li key={i} className="jogo">
          <div className="dezenas">
            {jogo.map((n) => <Bola key={n} numero={n} selecionada />)}
          </div>
          <button onClick={() => remover(i)} aria-label="Remover jogo">🗑</button>
        </li>
      ))}
    </ul>
  );
}
```

8.2 Reaproveitando a `Bola`

Note que reusamos o átomo `Bola` (do Módulo 2) para mostrar as dezenas de cada jogo salvo — a mesma peça da grade de seleção, agora só para exibir. É a vantagem do Atomic Design: uma peça bem feita serve a vários contextos. A `Bola` não sabe se está numa grade de seleção ou numa lista de jogos; ela só desenha um número.

8.3 O padrão "mudar o dado, a tela reage"

Ao remover um jogo, fazemos duas coisas: persistir a remoção (`removerJogo`) e **atualizar o estado** (`setJogos(...)`). É a atualização do estado que faz a lista redesenhar sem o item. Compare com o app vanilla original, que removia o `<li>` do DOM na mão e ainda tinha que "reindexar" os itens guardados — uma lógica frágil e cheia de bugs. Em React, você muda o dado, e o React cuida da tela.

> **IMPORTANTE — A `KEY` NUMA LISTA QUE MUDA**
>
> Aqui usamos `key={i}` (o índice) por simplicidade, mas com uma ressalva: como esta lista **pode ser reordenada** ao remover um item, o ideal seria dar a cada jogo um id único ao salvá-lo (ex.: um timestamp) e usar esse id como key. Índice como key é aceitável em listas que só crescem no fim, mas em listas que removem do meio, ele pode causar bugs visuais (o React reaproveita o componente errado). Ao evoluir o app, dê ids aos jogos — é o jeito certo.

8.4 Excluir todos

Um botão para limpar tudo, com confirmação (para não apagar por engano):

```
import { excluirTodos } from '../../services/storage';

function limparTudo() {
  if (!confirm('Remover TODOS os jogos desta loteria?')) return;
  excluirTodos(config.url);
  setJogos([]);
}
```

> **CUIDADO — AÇÕES DESTRUTIVAS PEDEM CONFIRMAÇÃO**
>
> "Excluir todos" apaga dados do usuário para sempre. Sempre confirme antes de ações irreversíveis — um simples `confirm()` do navegador já evita a tragédia de um clique acidental limpar horas de jogos montados. Para um app mais polido, um modal customizado é melhor que o `confirm()` nativo, mas o princípio é o mesmo: dê ao usuário uma chance de recuar antes do irreversível.

8.5 Fecho do módulo

O fluxo central está completo: o usuário navega até uma loteria, monta jogos numa grade interativa, salva, lista e remove. O LMania já é útil. Mas os jogos ainda precisam **sobreviver** ao fechar o navegador, e o app precisa funcionar **sem internet** — é a persistência offline, o coração do próximo módulo. E é lá que os jogos que "salvamos" ganham um lar de verdade.

Módulo 4 — Dados, persistência e offline

O LMania é "offline-first": os jogos vivem no dispositivo do usuário e o app funciona sem internet. Neste módulo você aprende a persistir dados localmente de forma limpa, a buscar resultados oficiais com cache, a conferir jogos, e a transformar o site num app instalável (PWA) que abre sem rede.

Capítulo 9 — Persistência local com localStorage

9.1 O localStorage como "disco"

O `localStorage` do navegador guarda dados que sobrevivem ao fechamento — é o "disco" do app. Mas acessá-lo direto, espalhado pela interface, vira bagunça. Encapsulamos tudo num **service** com uma API limpa; os componentes chamam funções nomeadas e não sabem os detalhes.

```
// src/services/storage.js — puro, sem React
const chave = (loteria) => `lmania_jogos_${loteria}`;

export function listarJogos(loteria) {
  try { return JSON.parse(localStorage.getItem(chave(loteria))) ?? []; }
  catch { return []; }
}
export function salvarJogo(loteria, dezenas) {
  const jogos = listarJogos(loteria);
  salvar(loteria, [...jogos, dezenas]);       // array novo — imutável
}
export function removerJogo(loteria, indice) {
  salvar(loteria, listarJogos(loteria).filter((_, i) => i !== indice));
}
export function excluirTodos(loteria) { localStorage.removeItem(chave(loteria)); }

function salvar(loteria, jogos) {
  localStorage.setItem(chave(loteria), JSON.stringify(jogos));
}
```

Um array por loteria (JSON) é simples: adicionar é `[...jogos, novo]`, remover é `filter`. Sem "contadores" nem "reindexação" como no código antigo — a complexidade some.

9.2 Um hook para casar estado e disco

O padrão "estado em memória + cópia no disco" (que vimos ao salvar jogos) merece um hook reutilizável — para tema, preferências, qualquer coisa que persista:

```
// src/hooks/useLocalStorage.js
import { useState, useEffect } from 'react';

export function useLocalStorage(chave, inicial) {
  const [valor, setValor] = useState(() => {
    try { const s = localStorage.getItem(chave); return s ? JSON.parse(s) : inicial; }
    catch { return inicial; }
  });
  useEffect(() => { localStorage.setItem(chave, JSON.stringify(valor)); }, [chave, valor]);
  return [valor, setValor];
}
```

Agora `const [tema, setTema] = useLocalStorage('lmania_tema', 'escuro')` — o `tema` reage como estado e persiste sozinho.

> **IMPORTANTE — ESTADO EM MEMÓRIA, DISCO EM PARALELO**
>
> O princípio: mantenha o dado no **estado** (para a tela reagir na hora) e escreva no `localStorage` **em paralelo** (para sobreviver ao fechamento). Nunca faça a tela ler do disco a cada render — leia uma vez na inicialização, e depois o estado é a verdade. O `useLocalStorage` encapsula exatamente esse padrão, e você o reusa o app inteiro.

9.3 Migração de dados: não perca os jogos dos usuários

Se você está reconstruindo um app que já tem usuários (como o LMania original, com jogos salvos num formato antigo), é **responsabilidade sua** não fazê-los perder os dados. Escreva uma migração que roda uma vez: lê o formato antigo, converte, e salva no novo.

```
// src/services/storage.js — roda uma vez na inicialização do app
export function migrarFormatoAntigo() {
  if (localStorage.getItem('migrado_v2')) return;   // já migrou
  // ...lê as chaves antigas (contador + itens numerados), converte para arrays...
  localStorage.setItem('migrado_v2', 'true');
}
```

> **CUIDADO — MIGRAÇÃO DE DADOS É COISA SÉRIA**
>
> Mudar o formato de armazenamento sem migrar faz os usuários abrirem a versão nova e "perderem" tudo que salvaram. Isso destrói a confiança. Sempre que mudar o esquema de dados, escreva a migração, teste-a com dados reais do formato antigo, e só então publique. Nunca quebre o que já está no dispositivo das pessoas — é um princípio ético de quem mantém software de verdade.

Com a persistência resolvida, o próximo passo é trazer dados de fora — os resultados oficiais das loterias — para conferir os jogos.

Capítulo 10 — Buscando resultados e conferindo jogos

10.1 A API de resultados oficiais

Para conferir jogos, precisamos do resultado oficial de um concurso. As loterias da Caixa têm uma API pública. Criamos um service que busca e — importante — **cacheia** o resultado, porque um concurso já sorteado é imutável (não muda nunca):

```
// src/services/resultados.js
export async function buscarResultado(loteria, concurso) {
  const chave = `resultado-${loteria}-${concurso}`;
  const cache = localStorage.getItem(chave);
  if (cache) return JSON.parse(cache);        // já temos: instantâneo, sem rede

  const url = `https://servicebus2.caixa.gov.br/portaldeloterias/api/${loteria}/${concurso}`;
  const r = await fetch(url);
  if (!r.ok) throw new Error('Concurso não encontrado.');
  const dados = await r.json();
  localStorage.setItem(chave, JSON.stringify(dados)); // guarda para sempre
  return dados;
}
```

DICA — CACHE PARA DADOS IMUTÁVEIS

Um resultado de concurso passado nunca muda. Então buscá-lo **uma vez** e servir do cache depois é a estratégia perfeita: economiza rede, funciona offline (depois da primeira busca), e é instantâneo. A regra geral: dados que não mudam devem ser cacheados agressivamente; só dados que mudam (o próximo sorteio ainda não realizado) precisam ir à rede sempre.

CUIDADO — O CORS E APIS DE TERCEIROS

A API da Caixa é de terceiros — o navegador pode bloquear a chamada por **CORS** (uma regra de segurança). Não é bug do seu código; quem libera é o servidor. Se acontecer em desenvolvimento, use o **proxy do Vite** (`server.proxy` no `vite.config.js`) para repassar a chamada. Em produção, pode ser necessário um pequeno proxy no seu próprio servidor. É uma dor de cabeça clássica de consumir APIs que você não controla.

10.2 Conferindo: comparar jogo com resultado

Conferir é lógica pura — um service que recebe um jogo e as dezenas sorteadas, e conta os acertos:

```
// src/services/loterias/conferencia.js
export function conferir(jogo, dezenasSorteadas) {
  const sorteadas = new Set(dezenasSorteadas);
  const acertos = jogo.filter((n) => sorteadas.has(n));
  return { acertos: acertos.length, dezenasAcertadas: acertos };
}
```

Simples e testável: dado `[5, 12, 23, 34]` e sorteio `[5, 34, 40, ...]`, devolve 2 acertos. Nenhuma UI, nenhum React — só a regra.

10.3 A tela de conferência

A tela junta as peças: busca o resultado, confere cada jogo salvo, e mostra os acertos com destaque visual:

```
// src/pages/Loteria/Conferir.jsx (essência)
const [concurso, setConcurso] = useState('');
const [resultado, setResultado] = useState(null);
const [carregando, setCarregando] = useState(false);
const [erro, setErro] = useState(null);

async function conferirTudo() {
  setCarregando(true); setErro(null);
  try {
    const r = await buscarResultado(config.url, concurso);
    setResultado(r);
  } catch (e) {
    setErro(e.message);
  } finally {
    setCarregando(false);
  }
}

// ao ter o resultado, confere cada jogo salvo:
const jogos = listarJogos(config.url);
const conferidos = resultado
  ? jogos.map((jogo) => ({ jogo, ...conferir(jogo, resultado.dezenas) }))
  : [];
```

E a renderização trata os quatro estados (o quarteto de sempre): carregando, erro, sem resultado ainda, e conferidos — mostrando cada jogo com suas dezenas acertadas em destaque.

> **IMPORTANTE — A LÓGICA SEPARADA DA TELA**
>
> Note a divisão: `buscarResultado` e `conferir` são services **puros** (sem React); a tela só os orquestra e desenha o resultado. Isso permite testar a conferência de forma isolada — você garante que "jogo com 4 das 6 dezenas certas = 4 acertos" sem montar tela nenhuma. Quando a lógica é pura e a tela é fina, o app fica confiável e fácil de evoluir. É a arquitetura em camadas, de novo, pagando dividendos.

O app já busca resultados e confere jogos. Falta a promessa do "offline-first": funcionar sem internet e ser instalável. É o PWA — o próximo capítulo.

Capítulo 11 — PWA: instalável e offline

11.1 O que é um PWA

Um **PWA** (*Progressive Web App*) é um site que se comporta como um app nativo: pode ser **instalado** na tela inicial do celular (com ícone próprio, sem a barra do navegador), abre em tela cheia e funciona **offline**. Para o LMania — uma ferramenta que o usuário abre toda hora e que faz quase tudo localmente — isso é ideal.

Dois ingredientes transformam o site em PWA: um **manifesto** (a identidade do app) e um **service worker** (o cache inteligente).

11.2 O jeito fácil: vite-plugin-pwa

Escrever um service worker à mão é trabalhoso e cheio de armadilhas de versão. O plugin `vite-plugin-pwa` gera tudo:

```
npm install -D vite-plugin-pwa
```

```
// vite.config.js
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',   // atualiza o app sozinho quando você publica
      manifest: {
        name: 'LMania — Loterias',
        short_name: 'LMania',
        display: 'standalone',       // sem a barra do navegador
        background_color: '#080c14',
        theme_color: '#f97316',
        icons: [
          { src: '/icons/192.png', sizes: '192x192', type: 'image/png' },
          { src: '/icons/512.png', sizes: '512x512', type: 'image/png' },
        ],
      },
      workbox: {
        globPatterns: ['**/*.{js,css,html,png,json}'], // cacheia o "casco" do app
      },
    }),
  ],
});
```

O `display: 'standalone'` remove a barra de endereço; os ícones aparecem na tela inicial. E o `workbox` cacheia os arquivos do app, fazendo-o abrir instantâneo e funcionar sem rede.

11.3 O que cachear (e o que não)

> **IMPORTANTE — CACHEIE O CASCO, NÃO OS DADOS VOLÁTEIS**
>
> Cacheie o **casco** do app (código, estilos, ícones) — muda pouco e faz o app abrir offline. **Não** cacheie dados que precisam ser frescos. No LMania há uma distinção elegante: resultados de concursos **passados** são imutáveis (já ficam no localStorage, cacheados para sempre); o app é cacheado pelo PWA; e os cálculos (fechamento, combinações) rodam localmente, sem rede. Por isso o LMania faz quase tudo offline — só buscar um resultado **novo** precisa de internet.

11.4 O que funciona offline no LMania

Pense no que o usuário consegue fazer sem internet, graças à arquitetura que construímos:

- **Montar e salvar jogos** — tudo no localStorage.
- **Listar e remover jogos** — dados locais.
- **Fazer fechamentos e combinações** — os algoritmos rodam no dispositivo (Módulo 5).
- **Conferir concursos já buscados** — resultados no cache.
- **Ver estatísticas** dos jogos salvos.

O que **não** funciona offline: buscar um resultado **novo** (concurso recém-sorteado que ainda não foi cacheado). É uma degradação graciosa — o app não "quebra" sem rede, só faz um pouco menos.

> **DICA — A RECOMPENSA DA ARQUITETURA EM CAMADAS**
>
> Repare por que o fechamento funciona offline: porque a lógica está em `services/` puros, alimentada por dados locais, sem depender da rede. A separação em camadas (Módulo 2) e a decisão de manter os dados localmente se pagam aqui — de graça. Boas decisões de arquitetura rendem juros em lugares que você não previu, como o offline.

11.5 Instalação e o botão "Instalar"

O navegador oferece a instalação automaticamente (um ícone na barra), mas você pode oferecer um botão próprio, capturando o evento `beforeinstallprompt`:

```
// hook simplificado
const [promptInstall, setPromptInstall] = useState(null);
useEffect(() => {
  const handler = (e) => { e.preventDefault(); setPromptInstall(e); };
  window.addEventListener('beforeinstallprompt', handler);
  return () => window.removeEventListener('beforeinstallprompt', handler);
}, []);
// botão: {promptInstall && <Botao texto="⬇ Instalar" aoClicar={() => promptInstall.prompt()} />}
```

11.6 Fecho do módulo

O LMania agora é offline-first e instalável: persiste jogos, busca e cacheia resultados, confere, e funciona sem rede. É um app de verdade no dispositivo do usuário. Falta a parte que dá profundidade técnica ao livro — os **algoritmos** que fazem a ferramenta valer a pena: combinações e o fechamento matemático. É o próximo módulo, o mais interessante.

Módulo 5 — Os algoritmos da loteria

Aqui o livro fica sério — e mais interessante. A ferramenta de loterias esconde matemática de verdade: combinações e fechamentos. Vamos entendê-las **no papel, com diagramas**, antes de escrever o código, e vê-las virarem services puros. Este é o módulo que transforma "mais um app de CRUD" numa ferramenta com inteligência.

Capítulo 12 — Combinações: a base da matemática lotérica

12.1 A pergunta fundamental

Toda estratégia de loteria começa com uma pergunta: de quantas formas dá para escolher `k` dezenas dentre `n`? Isso são as **combinações**. Se você escolheu **8** dezenas favoritas e quer jogar todos os grupos de 6 (Mega-Sena), quantos jogos são? A resposta é a combinação de 8 tomados 6 a 6 — C(8,6) — que dá **28**.

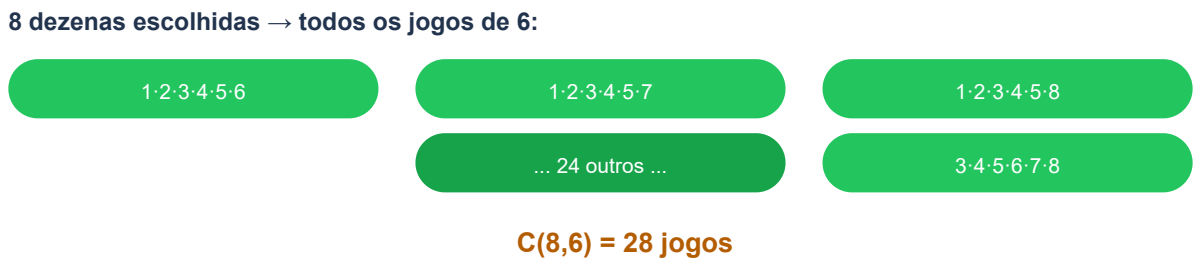


Figura 12.1 — Escolher 8 dezenas e jogar todos os grupos de 6 gera 28 jogos.

12.2 Gerando as combinações no código

O gerador é uma função recursiva clássica. A ideia: "para formar um jogo de `k` dezenas, escolho uma e formo um jogo de `k-1` com as que sobraram **à direita** dela" (olhar só à direita evita repetir o mesmo jogo em ordem diferente).

```
// src/services/loterias/combinacoes.js — puro, testável
export function gerarCombinacoes(dezenas, tamanho, limite = Infinity) {
  const resultados = [];
  function aux(atual, restante, faltam) {
    if (resultados.length >= limite) return;
    if (faltam === 0) { resultados.push(atual); return; }   // jogo completo
    for (let i = 0; i <= restante.length - faltam; i++) {
      aux(atual.concat(restante[i]), restante.slice(i + 1), faltam - 1);
    }
  }
  aux([], dezenas, tamanho);
  return resultados;
}
// gerarCombinacoes([1,2,3,4,5,6,7,8], 6) → 28 jogos
```

NOTA — COMO A RECURSÃO "QUEBRA" O PROBLEMA

A função resolve um problema grande dividindo em menores: escolher 6 de 8 vira "escolho a primeira, e escolho 5 das 7 restantes à direita", e assim por diante, até `faltam === 0` (um jogo completo). O `restante.slice(i + 1)` — só olhar à direita — garante que `[1,2]` e `[2,1]` (o mesmo jogo) não sejam contados duas vezes. Esse padrão "dividir para conquistar" aparece em muitos algoritmos; reconhecê-lo é uma habilidade transferível.

12.3 O `limite`: cuidado com a explosão combinatória

Repare no parâmetro `limite`. Ele existe por um motivo sério: as combinações **explodem**. C(8,6) são 28, mas C(20,6) são 38.760, e C(25,15) (Lotofácil) são **3.268.760**. Gerar todas travaria o navegador. O `limite` corta a geração num teto seguro.

CUIDADO — A EXPLOSÃO COMBINATÓRIA TRAVA A TELA

Combinações crescem absurdamente rápido. Gerar "todas as combinações" de muitas dezenas pode gerar milhões de jogos e congelar o navegador (lembra que o JavaScript roda numa thread só, que também desenha a tela). Duas defesas: o `limite` (nunca gere mais que um teto), e — para cálculos genuinamente pesados — mover o trabalho para um **Web Worker** (uma thread paralela que não trava a interface). No Vite: `import Worker from './calc.worker?worker'`. Veremos a ideia no fechamento.

12.4 Selecionando um subconjunto

Como jogar todas as combinações é caro, um recurso comum é gerar todas (até o limite) e **sortear** um subconjunto — por exemplo, "me dê 10 jogos aleatórios entre todas as combinações das minhas dezenas":

```
export function selecionarAleatorias(combinacoes, quantidade) {
  const indices = new Set();
  while (indices.size < Math.min(quantidade, combinacoes.length)) {
    indices.add(Math.floor(Math.random() * combinacoes.length));
  }
  return [...indices].map((i) => combinacoes[i]);
}
```

Isso já é útil, mas é "burro" — os jogos sorteados podem se sobrepor mal, desperdiçando cobertura. A forma **inteligente** de reduzir jogos mantendo garantia de prêmio é o **fechamento** — e é um problema muito mais interessante, o assunto do próximo capítulo.

IMPORTANTE — A LÓGICA PURA MORA NO SERVICE

Todo este capítulo é `services/loterias/combinacoes.js` — JavaScript puro, sem React. A tela apenas chama `gerarCombinacoes(...)` e mostra o resultado. Isso torna a matemática testável (você verifica que C(8,6) dá exatamente 28) e reutilizável (a mesma função serve à tela de combinações e ao fechamento). Separar o cálculo da apresentação é o que mantém algoritmos complexos sob controle — e é a lição que se aplica a qualquer app com lógica pesada.

Você domina as combinações. Agora o algoritmo estrela: o fechamento, que joga poucos jogos com garantia matemática.

Capítulo 13 — Fechamento: jogar poucos com garantia

13.1 O dilema do custo

Jogar todas as combinações das suas dezenas garante o prêmio se elas saírem — mas é caro. Com 8 dezenas na Mega, são 28 jogos (R$ 168). Com 15, são 5005 jogos (R$ 30.030). Impraticável.

O **fechamento** (ou desdobramento) resolve o dilema: em vez de jogar *todas* as combinações, jogar um **subconjunto inteligente** que ainda **garante** certo prêmio se um número de dezenas for acertado. Por exemplo: "com minhas 15 dezenas, quero jogar só 30 jogos, mas de forma que, se 4 das minhas 15 saírem, eu tenha garantidamente pelo menos uma quadra".

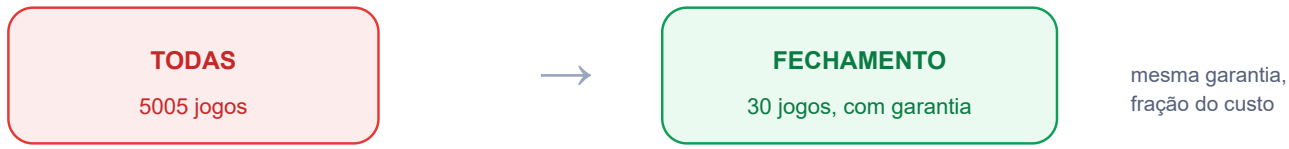


Figura 13.1 — O fechamento troca "jogar tudo" por "jogar poucos, com garantia matemática". É o coração da ferramenta.

13.2 O que é a "garantia" — problemas de cobertura

Matematicamente, um fechamento é um **problema de cobertura**. Pense assim: cada jogo de 6 dezenas "cobre" todas as quadras (grupos de 4) que estão dentro dele. Queremos escolher **poucos** jogos que, juntos, **cubram todas** as quadras possíveis das nossas dezenas. Porque se qualquer quadra sair, algum dos nossos jogos a contém — e o prêmio da quadra está garantido.

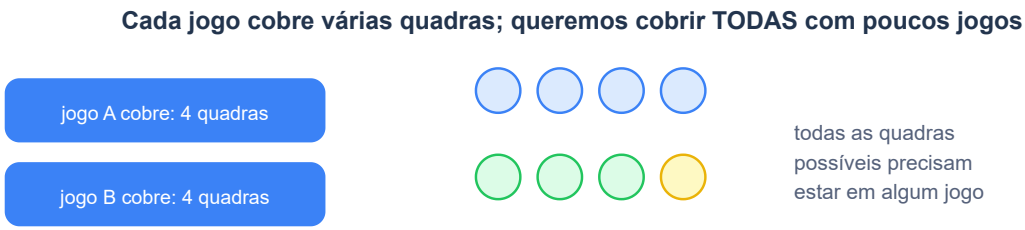


Figura 13.2 — Fechamento é cobrir todas as quadras possíveis com o menor número de jogos. Achar o mínimo é um problema difícil.

13.3 Por que não calcular na hora — a decisão de engenharia

Achar o **menor** conjunto que cobre tudo é um problema **NP-difícil**: não existe fórmula rápida para o ótimo, e calcular na hora, para muitas dezenas, travaria o navegador. Por isso a ferramenta **não calcula o fechamento em tempo real**. Ela usa **coberturas pré-calculadas** — tabelas geradas antes (por um algoritmo guloso, *greedy*, que monta uma solução boa, próxima do mínimo) e guardadas como dado.

```
// src/services/loterias/coberturas.js  → SÓ DADOS (a tabela pré-calculada)
// mapa: "quantasDezenas_garantia" → lista de "padrões" (índices de dezenas por jogo)
export const COBERTURAS = {
  '8_4': [[0,1,2,3,4,5], [0,1,2,3,6,7], /* ...os padrões... */],
  // ...uma entrada por combinação (nº de dezenas, garantia)
};
```

IMPORTANTE — A DECISÃO A PRESERVAR

Este é o ponto mais importante do capítulo. O fechamento **não é calculado ao vivo** — ele consulta tabelas prontas. É uma decisão consciente de engenharia: o cálculo ótimo é NP-difícil e travaria a tela. A arquitetura correta é: as **coberturas** viram um asset de dados (`coberturas.js`), e a **lógica de fechar** vira um service puro que aplica a tabela às dezenas do usuário. Nunca tente "melhorar" recalculando na hora — você quebraria a performance. Às vezes, a decisão mais inteligente é **não calcular**.

13.4 Aplicando a tabela: o service de fechamento

A lógica de fechar é simples, porque o trabalho pesado já foi feito na tabela. O service pega os "padrões" (que são índices) e os aplica às dezenas reais que o usuário escolheu:

```
// src/services/loterias/fechamento.js — puro
import { COBERTURAS } from './coberturas';

export function fechar(dezenasEscolhidas, garantia) {
  const chave = `${dezenasEscolhidas.length}_${garantia}`;
  const padroes = COBERTURAS[chave];
  if (!padroes) return null;   // não há cobertura para essa combinação
  // cada padrão é uma lista de índices; troca índice pela dezena real
  return padroes.map((indices) => indices.map((i) => dezenasEscolhidas[i]));
}
// fechar([5,12,23,34,41,52,7,18], 4) → poucos jogos que garantem a quadra
```

13.5 A tela de fechamento

O organismo React só cuida da interface — a grade de seleção, a escolha da garantia, e a exibição dos jogos gerados — chamando o service:

```jsx
// components/organisms/Fechar/Fechar.jsx (essência)
const [dezenas, setDezenas] = useState(new Set());
const [garantia, setGarantia] = useState(4);
const [jogos, setJogos] = useState([]);

function gerar() {
  const resultado = fechar([...dezenas].sort((a,b)=>a-b), garantia);
  if (resultado) setJogos(resultado);
  else toast.error('Sem cobertura para essa combinação de dezenas e garantia.');
}
```

DICA — SE UM DIA PRECISAR CALCULAR AO VIVO, USE UM WORKER

Para os casos das tabelas, o cálculo é instantâneo (só aplicar índices). Mas se você quisesse gerar coberturas na hora (para combinações não tabeladas), aí sim o trabalho seria pesado — e você o moveria para um **Web Worker**: uma thread paralela que calcula sem congelar a tela. Como o `fechar` é um service puro, chamá-lo de dentro de um worker é trivial, sem tocar na UI. É a recompensa, mais uma vez, de manter a lógica separada da interface.

13.6 Fecho do módulo... quase

Você entendeu e implementou os dois algoritmos que dão inteligência à ferramenta: combinações (a base) e fechamento (a otimização com garantia). Falta um toque que transforma dados em insight visual: as **estatísticas com gráficos** — o próximo e último capítulo do módulo.

Capítulo 14 — Estatísticas e gráficos

14.1 Transformando dados em insight

Números crus dizem pouco; um gráfico diz muito. A ferramenta mostra estatísticas dos jogos e dos resultados — quantas dezenas pares vs. ímpares, quais dezenas saem mais. Primeiro, o cálculo (service puro); depois, a visualização (componente).

14.2 O cálculo: pares e ímpares

Uma estatística simples e popular: a distribuição de dezenas pares e ímpares num jogo ou num conjunto de resultados. Lógica pura:

```
// src/services/loterias/estatisticas.js
export function contarParesImpares(dezenas) {
  let pares = 0, impares = 0;
  for (const n of dezenas) (n % 2 === 0 ? pares++ : impares++);
  return { pares, impares };
}

// frequência de cada dezena num histórico de resultados
export function frequenciaDezenas(historicoDeSorteios) {
  const freq = {};
  for (const sorteio of historicoDeSorteios) {
    for (const n of sorteio) freq[n] = (freq[n] ?? 0) + 1;
  }
  return freq; // { 5: 12, 23: 9, ... }
}
```

14.3 A biblioteca de gráficos

Em vez de desenhar gráficos na mão, usamos uma biblioteca pronta e amigável ao React. Uma boa escolha é o **recharts** (`npm install recharts`), que oferece componentes de gráfico declarativos:

```jsx
// src/components/organisms/GraficoParesImpares/GraficoParesImpares.jsx
import { PieChart, Pie, Cell, Legend, ResponsiveContainer } from 'recharts';
import { contarParesImpares } from '../../../services/loterias/estatisticas';

export default function GraficoParesImpares({ dezenas }) {
  const { pares, impares } = contarParesImpares(dezenas);
  const dados = [
    { nome: 'Pares', valor: pares },
    { nome: 'Ímpares', valor: impares },
  ];
  const cores = ['#3b82f6', '#f97316'];

  return (
    <ResponsiveContainer width="100%" height={240}>
      <PieChart>
        <Pie data={dados} dataKey="valor" nameKey="nome" outerRadius={90} label>
          {dados.map((_, i) => <Cell key={i} fill={cores[i]} />)}
        </Pie>
        <Legend />
      </PieChart>
    </ResponsiveContainer>
  );
}
```

Repare no padrão familiar: o **service puro** calcula (`contarParesImpares`), e o **componente** apenas visualiza. O gráfico é declarativo — você descreve os dados e as cores, e a biblioteca desenha. É o modelo React aplicado a gráficos.

> **DICA — BIBLIOTECA DECLARATIVA COMBINA COM REACT**
>
> Bibliotecas como o recharts são "declarativas": você passa os dados como props e ela desenha, reagindo a mudanças. Isso combina com o React — quando os dados mudam, o gráfico se atualiza sozinho, como qualquer componente. Compare com bibliotecas imperativas (como o Chart.js puro, do app original), onde você cria o gráfico e o atualiza na mão, com cuidado para não vazar memória. Ao usar React, prefira bibliotecas que abraçam o modelo declarativo.

14.4 O `ResponsiveContainer`

Note o `<ResponsiveContainer>` envolvendo o gráfico. Ele faz o gráfico se adaptar ao tamanho da tela — essencial num app que roda no celular e no desktop. É o mesmo espírito de responsividade que aplicamos ao layout: um código, várias telas.

IMPORTANTE — O PADRÃO QUE SE REPETIU O LIVRO INTEIRO

Pela última vez, note a divisão: o cálculo (`estatisticas.js`) é puro e testável; o componente (`GraficoParesImpares`) só visualiza. Esse padrão — **lógica no service, apresentação no componente** — apareceu em cada módulo: nos jogos, na conferência, no fechamento, e agora nos gráficos. Não é coincidência: é a arquitetura que torna o app inteiro compreensível, testável e evoluível. Se você levar uma lição deste livro, que seja esta.

14.5 Fecho do módulo

Você deu inteligência à ferramenta: combinações, fechamento matemático e estatísticas visuais — tudo com lógica pura nos services e apresentação nos componentes. O LMania agora é uma ferramenta completa e poderosa. Falta o acabamento de um produto público: conteúdo (blog), monetização (anúncios), SEO e publicação. É o **Módulo 6**, o final.

Módulo 6 — Conteúdo e publicação

A ferramenta está pronta; falta transformá-la em produto público. Neste módulo final adicionamos conteúdo (um blog, que traz visitantes e ajuda no SEO), o espaço de anúncios (padronizado com outros apps da casa), e publicamos o LMania no mundo — instalável, rápido e no ar.

Capítulo 15 — Um blog dentro do app

15.1 Por que um blog

Um blog de conteúdo (artigos sobre probabilidade, estatística, estratégias) traz visitantes pelos buscadores e dá autoridade ao app. Ele exemplifica dois padrões importantes que ainda não vimos juntos: **buscar uma lista** e **abrir um item** por uma rota dinâmica.

15.2 O service do blog

Como sempre, a comunicação com o servidor mora num service:

```
// src/services/blog.js
const BLOG_BASE = import.meta.env.VITE_BLOG_API ?? 'https://api.lmania.com.br/blog';

export const listarPosts = () => fetch(`${BLOG_BASE}/posts`).then((r) => r.json());
export const buscarPost = (slug) => fetch(`${BLOG_BASE}/posts/${slug}`).then((r) => r.json());
```

15.3 A lista de posts

A tela de blog busca os posts e os lista com links para cada um:

```jsx
// src/pages/Blog/Blog.jsx
import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { listarPosts } from '../../services/blog';
import { useSeo } from '../../hooks/useSeo';

export default function Blog() {
  useSeo('Blog | LMania', 'Artigos sobre loterias, probabilidade e estatística.');
  const [posts, setPosts] = useState([]);
  const [carregando, setCarregando] = useState(true);

  useEffect(() => {
    let ativo = true;
    listarPosts()
      .then((d) => { if (ativo) setPosts(d); })
      .finally(() => { if (ativo) setCarregando(false); });
    return () => { ativo = false; };
  }, []);

  if (carregando) return <p>Carregando artigos...</p>;
  return (
    <div>
      <h2>Blog</h2>
      {posts.map((post) => (
        <Link key={post.slug} to={`/blog/${post.slug}`} className="card-post">
          <h3>{post.titulo}</h3>
          <p>{post.resumo}</p>
        </Link>
      ))}
    </div>
  );
}
```

15.4 O post individual: rota dinâmica

Cada post tem sua própria URL (`/blog/como-funciona-o-fechamento`). A tela lê o `slug` da rota e busca o conteúdo:

```jsx
// src/pages/BlogPost/BlogPost.jsx
import { useParams } from 'react-router-dom';
import { useState, useEffect } from 'react';
import { buscarPost } from '../../services/blog';
import { useSeo } from '../../hooks/useSeo';

export default function BlogPost() {
  const { slug } = useParams();              // o trecho variável da URL
  const [post, setPost] = useState(null);

  useEffect(() => {
    buscarPost(slug).then(setPost);
  }, [slug]);                                // rebusca se o slug mudar

  useSeo(post ? `${post.titulo} | LMania` : 'Carregando...', post?.resumo ?? '');

  if (!post) return <p>Carregando...</p>;
  return (
    <article>
      <h1>{post.titulo}</h1>
      <div dangerouslySetInnerHTML={{ __html: post.conteudoHtml }} />
    </article>
  );
}
```

CUIDADO — O `DANGEROUSLYSETINNERHTML`

Para renderizar HTML vindo do servidor (o corpo do post), usamos `dangerouslySetInnerHTML`. O nome é um aviso sério: se o HTML contiver scripts maliciosos, eles seriam executados na sua página (ataque XSS). Só use com conteúdo de uma fonte **confiável** (o seu próprio backend) e, idealmente, sanitizado no servidor. Nunca injete HTML vindo diretamente de um usuário sem limpá-lo. O nome "dangerously" é literal — respeite-o.

IMPORTANTE — O PADRÃO "LISTA → DETALHE"

O blog exemplifica um padrão universal: uma tela de **lista** (com links) e uma tela de **detalhe** (que lê um id/slug da rota e busca o item). Você o encontrará em quase todo app — produtos e detalhe de produto, usuários e perfil, posts e artigo. Dominar esse par (lista com `.map()` de `<Link>` + detalhe com `useParams` + busca) é dominar metade das telas que você vai construir na vida.

Com conteúdo pronto, adicionamos o espaço de anúncios — o item que padroniza o LMania com os outros apps da casa.

Capítulo 16 — O espaço de anúncios

16.1 Um banner padronizado entre apps

Apps gratuitos costumam ter anúncios, e é comum que vários apps da mesma casa compartilhem o **mesmo** sistema, gerenciado num servidor central. No LMania, os anúncios vêm de um endpoint compartilhado: `https://kibusao.com/public/api/anuncios.php`. Construímos um `BannerAnuncio` que consome esse endpoint — um componente projetado para ser **idêntico** entre apps.

16.2 O contrato do endpoint

O servidor devolve este JSON (o **contrato** — o formato que o componente espera):

```
{
  "habilitado": true,
  "intervaloMs": 6000,
  "anuncios": [
    { "id": "a1", "tipo": "texto", "titulo": "Anuncie aqui",
      "texto": "Sua marca vista por milhares.", "link": "https://...", "imagemUrl": "" }
  ]
}
```

Regras: se `habilitado` for falso ou a lista vier vazia, **não mostra nada**; com 2+ anúncios, rotaciona a cada `intervaloMs`; `tipo: "imagem"` mostra a imagem, senão o texto; havendo `link`, abre em nova aba.

16.3 O componente

```jsx
// src/components/BannerAnuncio/BannerAnuncio.jsx
import { useEffect, useState } from 'react';
import estilos from './BannerAnuncio.module.css';

const ANUNCIOS_URL = 'https://kibusao.com/public/api/anuncios.php';

export default function BannerAnuncio() {
  const [dados, setDados] = useState(null);
  const [indice, setIndice] = useState(0);

  useEffect(() => {
    let vivo = true;
    fetch(ANUNCIOS_URL)
      .then((r) => (r.ok ? r.json() : null))
      .then((d) => { if (vivo) setDados(d ?? { habilitado: false, anuncios: [] }); })
      .catch(() => { if (vivo) setDados({ habilitado: false, anuncios: [] }); });
    return () => { vivo = false; };
  }, []);

  const anuncios = dados?.habilitado ? (dados.anuncios ?? []) : [];

  useEffect(() => {
    setIndice(0);
    if (anuncios.length < 2) return undefined;
    const ms = Math.max(2000, dados?.intervaloMs ?? 6000);
    const t = setInterval(() => setIndice((i) => (i + 1) % anuncios.length), ms);
    return () => clearInterval(t);
  }, [anuncios.length, dados?.intervaloMs]);

  if (anuncios.length === 0) return null;   // sem anúncio, sem espaço vazio

  const a = anuncios[indice % anuncios.length];
  const conteudo = a.tipo === 'imagem' && a.imagemUrl
    ? <img className={estilos.imagem} src={a.imagemUrl} alt={a.titulo || 'Anúncio'} />
    : <div className={estilos.texto}><b>{a.titulo}</b><span>{a.texto}</span></div>;

  return (
    <div className={estilos.banner}>
      <span className={estilos.rotulo}>Publicidade</span>
      {a.link
        ? <a href={a.link} target="_blank" rel="noopener noreferrer nofollow">{conteudo}</a>
        : conteudo}
    </div>
  );
}
```

16.4 Posicionando no LMania

O banner fica no topo do conteúdo, na Home e nas telas de lista — sem sobrepor a grade de dezenas (que é interativa):

```
// no topo da Home ou das telas de lista
<BannerAnuncio />
```

IMPORTANTE — OS DETALHES DO LINK (SEGURANÇA E SEO)

Repare no `rel="noopener noreferrer nofollow"` do link. `noopener` / `noreferrer` impedem que o site de destino manipule sua página (segurança); `nofollow` diz aos buscadores para não passar autoridade de SEO ao link pago (padrão de publicidade). E o rótulo "Publicidade" é transparência com o usuário. São detalhes pequenos que só quem entende inclui — e que fazem diferença em segurança, SEO e confiança.

DICA — O MESMO BANNER EM VÁRIOS APPS

Este `BannerAnuncio` não sabe nada do LMania — só busca os anúncios e se desenha. Por isso ele é "plug-and-play": o mesmo componente encaixa em qualquer app da casa, consumindo o mesmo endpoint, sem uma linha de diferença. É a padronização em ação: um servidor central de anúncios, um componente idêntico, vários apps. Se surgirem mais apps, extraia-o para um pacote compartilhado, com uma única fonte de verdade.

O LMania está completo em funcionalidades e monetização. Falta o passo final: colocá-lo no mundo. É o deploy — o capítulo de encerramento.

Capítulo 17 — Build, deploy e o próximo passo

17.1 O build de produção

Para publicar, gere a versão otimizada — minificada e empacotada:

```
npm run build      # cria a pasta dist/ com arquivos estáticos
npm run preview    # testa o build localmente, como seria em produção
```

Sempre teste com `preview` antes de subir: muitos bugs (caminhos de imagem, variáveis de ambiente faltando) só aparecem no build real, não no modo dev.

17.2 Onde hospedar

Como o resultado são arquivos estáticos, as opções são muitas e boa parte gratuita:

- **Vercel / Netlify / Cloudflare Pages** — conecte o repositório do GitHub; a cada `git push`, eles constroem e publicam sozinhos, com HTTPS automático. O caminho mais fácil.
- **Servidor próprio** (Apache/Nginx) — suba o conteúdo de `dist/` e configure o servidor.

17.3 A pegadinha da SPA: o 404 nas rotas

Suas rotas (`/mega-sena`, `/blog/meu-post`) só existem no JavaScript. Se o usuário abrir uma delas **a frio** (digitando ou por link), o servidor procura o arquivo, não acha, e devolve **404**. A cura: configurar o servidor para devolver sempre o `index.html` em qualquer rota, deixando o React Router resolver.

```
# .htaccess (Apache) — devolve index.html para qualquer rota
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

Vercel e Netlify fazem isso automaticamente. Em servidor próprio, é você quem configura — e esquecer é o motivo nº 1 de "funciona no dev, links quebram em produção".

> **CUIDADO — HTTPS É OBRIGATÓRIO PARA O PWA**
>
> Publique sempre em **HTTPS**. Além de segurança, ele é *exigido* pelos recursos que o LMania usa: o service worker (o PWA offline/instalável) só funciona em HTTPS. Hoje o HTTPS é gratuito e automático nas plataformas modernas — não há desculpa. E aponte as variáveis de API (`VITE_...`) para os servidores reais em produção, não o `localhost` do desenvolvimento.

17.4 Mantendo o app vivo

Publicar é o começo da parte longa:

- **Dados envelhecem.** As tabelas de cobertura e os dados das loterias podem mudar; atualize e republique quando necessário. O service worker aplica a nova versão sozinho.
- **Monitore erros.** O `componentDidCatch` do Error Boundary (que você deve incluir, como no Provas) é o lugar de enviar erros para um serviço de monitoramento e descobrir bugs do mundo real antes dos usuários reclamarem.

17.5 Você chegou ao fim

> **NA PRÁTICA — O QUE VOCÊ CONSTRUIU**
>
> Você começou sem saber React e terminou com uma ferramenta real, publicada e instalável: grade interativa de dezenas, salvar/listar/conferir jogos, fechamentos matemáticos, estatísticas com gráficos, funcionamento offline, blog, temas e anúncios. Mais importante que o app: você aprendeu React de verdade — componentes e estado, arquitetura em camadas, persistência, PWA, algoritmos, gráficos e deploy — e tem o modelo mental para construir qualquer aplicação daqui em diante.

17.6 Para onde ir agora

- **TypeScript** — segurança de tipos; a migração num projeto Vite é gradual e indolor.
- **Testes** — seus services puros (combinações, fechamento, conferência) são perfeitos para testar com Vitest; comece por eles.
- **Web Workers** — se quiser calcular coberturas ao vivo, mova o trabalho pesado para uma thread paralela.
- **Um app seu, do zero** — é construindo que se aprende. Escolha uma ideia e aplique tudo isto.

A diferença entre copiar tutoriais e ser desenvolvedor está em entender *por que* cada peça existe. Ao longo deste livro, você construiu uma fundação, uma arquitetura, uma aplicação, uma inteligência (os algoritmos) e um produto — entendendo cada decisão. Agora o próximo app é seu. Construa. Entenda. E publique.